Day 133: Generative vs Discriminative Models: a Real Conceptual Comparison

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 1 | Complete study guide

Watch: (ML 1.5) Generative vs discriminative models (mathematicalmonk) | IAML2.23: Generative vs. discriminative learning (Victor Lavrenko)

Script: `day133_generative_vs_discriminative.py` · every number in this guide is copied by program from `day133_run_output.txt`, the real output of that script.

A discriminative classifier learns only where the boundary between classes runs. A generative one learns how each class's data is made, and finds the boundary by Bayes' rule. Both are written from scratch here, on MNIST and a medical table. With few training images the generative LDA wins (0.2922 error against 0.3702 at 100 images); with many, logistic regression does (0.0913 against 0.1278 at 60,000). Only the generative model can draw new digits, classify with half of every image hidden (0.7771 against 0.3923 for logistic regression with the hidden half filled with zeros), and notice pure noise: logistic regression is 1.0000 confident on it.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **discriminative** model learns p(class | x) directly: given an image, which digit? Logistic regression (Day 116's softmax regression) is one.

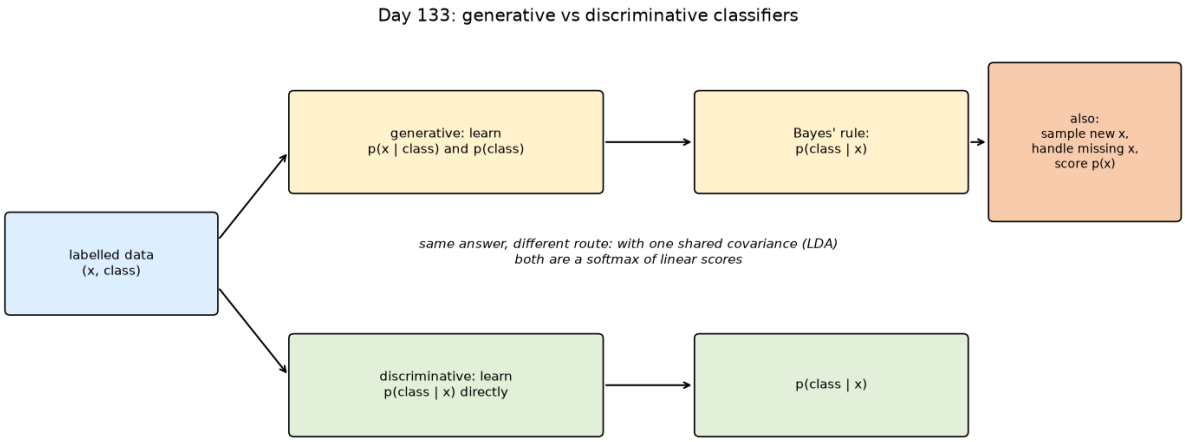
A **generative** model learns p(x | class), how images of each digit are spread, and p(class), how common each digit is. **Bayes' rule** turns them into p(class | x) = p(x | class) p(class) / p(x). Today's generative models put a **Gaussian** (a bell-shaped cloud with a mean and a covariance) on each class: **naive Bayes** (features independent), **LDA** (linear discriminant analysis: one covariance shared by all classes) and **QDA** (quadratic: one covariance per class).

Week 20 is about generating data. Day 114 compared naive Bayes with logistic regression on text; today the comparison is on images, and on the things only a generative model can do.

REAL-WORLD SCENARIO

Two ways to learn to tell apples from pears. One person learns what each fruit looks like, so they could also sketch one, or guess from half a photo. Another learns only the single feature that separates them best, quicker to learn well, but lost when shown a banana.

SYSTEM DIAGRAM



Two routes to p(class | x); only the generative route also models x itself.

Today's part for the week's build

`day133_exercise.py` is the Gaussian class models: fit, Bayes' rule, missing features, log p(x) and sampling, checked by hand. Its real output:

ACTUAL OUTPUT

```
log N(0; 0, 1): -0.918939; by hand -log(2 pi) / 2 = -0.918939
two 1-D classes: p(1 | x = 3.5) = 0.952574; by hand sigmoid(6 x 3.5 - 18) = 0.952574
LDA as linear scores against Bayes' rule: largest gap 6.1e-16
marginal (features 0 and 1 of a 3-feature model) against a model of those 2 features: largest gap
    0.0e+00
200,000 samples of class 1: mean off by 0.0009, covariance off by 0.0036
log p(x) of a typical point -3.287; of a far-away point -370.428
```

Data: MNIST (cached by Day 126): 60,000 training and 10,000 test images; and scikit-learn's bundled breast-cancer table (569 tumours, 30 measurements; benign or malignant), shuffled once into 369 training and 200 test rows. Day 116's softmax regression, Day 117's ROC AUC and Day 123's PCA are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: two Gaussians give a logistic posterior

DEFINITION — start here, no prior knowledge assumed

When two classes are Gaussians with the same spread, Bayes' rule gives p(class 1 | x) = sigmoid(w x + b): exactly the form logistic regression fits. The generative model gets w and b from the means and the variance; logistic regression fits them directly.

REAL-WORLD SCENARIO

Two piles of sand of the same shape: halfway between their peaks, a grain is equally likely to have come from either.

MATHEMATICS

$$p(1 \mid x) = p(x \mid 1)\, p(1) / [p(x \mid 0)\, p(0) + p(x \mid 1)\, p(1)] = \text{sigmoid}(w\, x + b)$$

$$w = (\mu_1 - \mu_0) / \sigma^2, \quad b = (\mu_0^2 - \mu_1^2) / (2\sigma^2) + \log(p(1) / p(0))$$

IMPLEMENTATION

```
def fit_gaussians(X, y, k, kind="shared", reg=1e-3):
    d = X.shape[1]; mu = np.array([X[y == c].mean(0) for c in range(k)])
    prior = np.bincount(y, minlength=k) / len(y)
    if kind == "shared":
        D = X - mu[y]; S = D.T @ D / len(X) + reg * np.eye(d)
        cov = np.repeat(S[None], k, 0)
    elif kind == "diagonal":
        cov = np.array([np.diag(X[y == c].var(0) + reg) for c in range(k)])
    else:
        cov = np.array([np.cov(X[y == c].T, bias=True).reshape(d, d) + reg * np.eye(d) for c in
            range(k)])
    return {"mu": mu, "cov": cov, "prior": prior}
def log_gauss(X, mu, S):
    L = np.linalg.cholesky(S); Z = np.linalg.solve(L, (X - mu).T)
    return -0.5 * ((Z ** 2).sum(0) + 2 * np.log(np.diag(L)).sum() + len(mu) * np.log(2 * np.pi))
def log_joint(model, X, keep=None):
    idx = np.arange(X.shape[1]) if keep is None else keep
    return np.stack([log_gauss(X[:, idx], m[idx], S[np.ix_(idx, idx)]) + np.log(p)
                     for m, S, p in zip(model["mu"], model["cov"], model["prior"])], 1)
def posterior(model, X, keep=None):
    J = log_joint(model, X, keep); J -= J.max(1, keepdims=True); P = np.exp(J)
    return P / P.sum(1, keepdims=True)
```

ACTUAL OUTPUT

```
means 1.0 and 5.0; shared variance (1/6) x (1 + 0 + 1 + 1 + 0 + 1) = 0.6667
by hand: p(class 1 | x) = sigmoid(w x + b) with w = (mu1 - mu0) / s2 = 6.0000, b = (mu0^2 - mu1^2)
    / (2 s2) = -18.0000
at x = 1, 3, 3.5: Bayes' rule gives 0.0000, 0.5000, 0.9526; the sigmoid gives 0.0000, 0.5000,
    0.9526
the boundary (probability 0.5) is halfway between the means, at x = 3: a straight line, as in
    logistic regression
Actual output from this lesson's run.
```

What it means

With means 1.0 and 5.0 and shared variance 0.6667, the formula gives w = 6.0000 and b = -18.0000, and Bayes' rule through the two Gaussians gives the same probabilities as the sigmoid: 0.5000 at x = 3 and 0.9526 at 3.5. A generative model with a shared spread draws a straight boundary.

WHY CHOOSE THIS?

Shows exactly how the two families are related.

WHY NOT THIS? (TRADEOFFS)

In one dimension; section 2 checks it on 784.

2. The same family, fitted two ways

DEFINITION — start here, no prior knowledge assumed

LDA on all 784 pixels (one covariance, slightly regularised) and logistic regression on the same pixels. Both end as a softmax of linear scores x W + b; they differ in how W and b are chosen. LDA: from means and a covariance, in one pass. Logistic regression: by gradient descent on the classification loss.

REAL-WORLD SCENARIO

Two maps of the same coast: one drawn from a survey of the land, one traced along the shoreline itself.

MATHEMATICS

$$\text{LDA: } w_c = \Sigma^{-1}\mu_c, \quad b_c = -\tfrac{1}{2}\mu_c\,\Sigma^{-1}\mu_c + \log p(c) \qquad p(c \mid x) = \text{softmax}(x\,W + b)$$

IMPLEMENTATION

```
def lda_linear(model):
    Si = np.linalg.inv(model["cov"][0]); W = Si @ model["mu"].T
    b = -0.5 * np.einsum("cd,dc->c", model["mu"], W) + np.log(model["prior"])
    return W, b
```

ACTUAL OUTPUT

```
LDA fitted in 0.9 s (means and one covariance: no iterations); test accuracy 0.8798
LDA's posterior as a softmax of linear scores x W + b: largest difference from Bayes' rule 8.0e-14
logistic regression fitted in 39.7 s (500 Adam steps); test accuracy 0.8917
   LDA       test log-loss 0.5019; average confidence 0.9353; share of predictions above 0.99 confident 0.6185
   logistic  test log-loss 0.9116; average confidence 0.9745; share of predictions above 0.99 confident 0.8431
they disagree on 0.1179 of test images
Actual output from this lesson's run.
```

What it means

The proof holds on real data: LDA's posterior and the softmax of its linear scores differ by at most 8.0e-14. Logistic regression is a little more accurate (0.8917 against 0.8798), but its stated confidence is worse: 0.8431 of its predictions claim more than 0.99 certainty, against 0.6185 for LDA, and its log-loss (which punishes confident mistakes) is 0.9116 against 0.5019. Its regularisation was light ($\lambda = 0.001$) and not tuned; whether stronger regularisation would close the gap was not tested.

WHY CHOOSE THIS?

LDA needs no iterations, and its probabilities were better calibrated here.

WHY NOT THIS? (TRADEOFFS)

Its Gaussian assumption costs accuracy once there is plenty of data.

3. Learning curves: little data against a lot

DEFINITION — start here, no prior knowledge assumed

Train each method on random subsets of growing size and measure test error. On MNIST each image is first reduced to 50 PCA numbers (Day 123, fitted without labels), so the covariances stay small. Ng and Jordan (2001) described the expected pattern: the generative model reaches its best error with fewer examples, the discriminative one reaches a lower error in the end.

REAL-WORLD SCENARIO

A student who memorises the textbook's summary is ready sooner; one who works through every past paper ends up sharper.

SPECIFICATION

Generative	naive Bayes and LDA with reg 0.1; QDA with reg 1.0
Discriminative	logistic regression, $\lambda = 0.001$, 500 Adam steps
Repeats	5 random subsets per size (MNIST), 20 (breast cancer); settings fixed, not re-tuned per size

IMPLEMENTATION

```
def learning_curve(fits, X, y, Xte, yte, sizes, draws, seed=0):
    rng = np.random.RandomState(seed); k = int(y.max()) + 1
    err = {m: np.zeros((len(sizes), draws)) for m in fits}
    for i, n in enumerate(sizes):
        for r in range(draws):
            while True:
                tr = rng.choice(len(X), n, replace=False)
                if np.bincount(y[tr], minlength=k).min() >= 2:
                    break
            for m, fit in fits.items():
                err[m][i, r] = np.mean(fit(X[tr], y[tr], k)(Xte).argmax(1) != yte)
    return err
```

ACTUAL OUTPUT

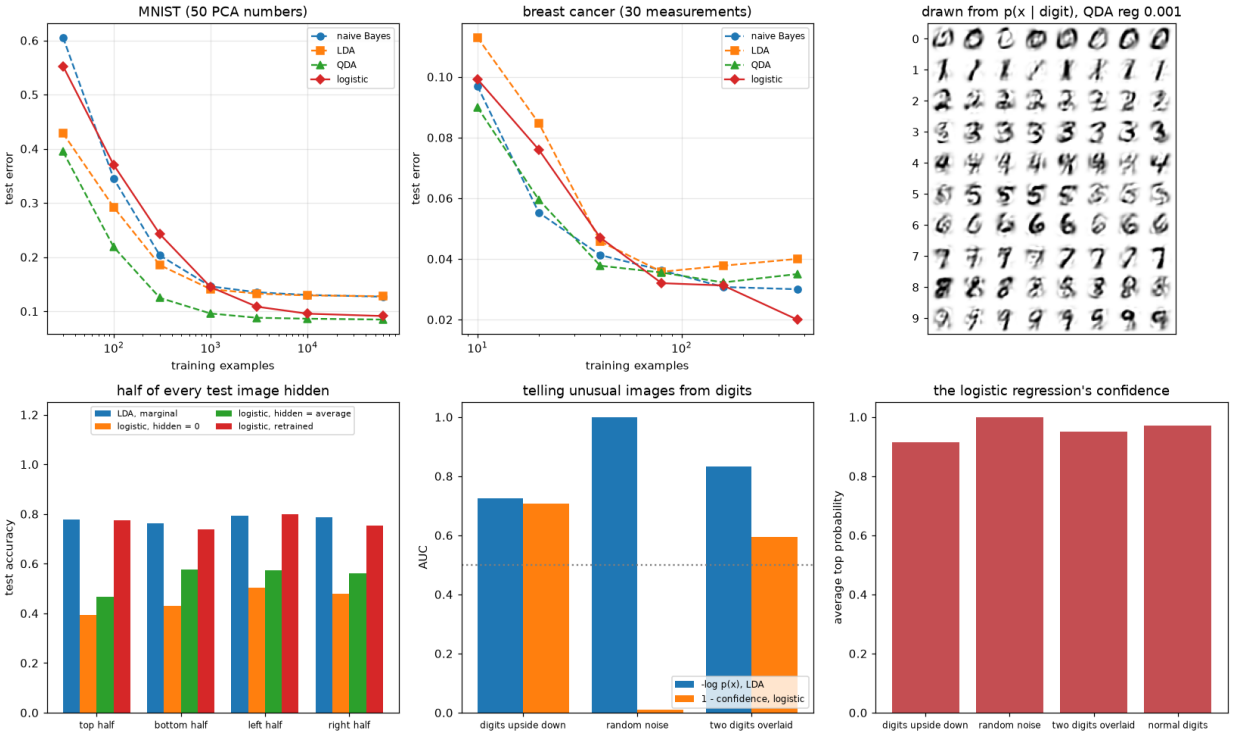
```
MNIST, 50 PCA numbers per image (PCA fitted on the 60,000 training images, without labels); 5 random subsets
    per size
   training images  naive Bayes          LDA          QDA     logistic   LDA - logistic
                30       0.6055       0.4285       0.3947       0.5524   -0.1239
               100       0.3447       0.2922       0.2185       0.3702   -0.0780
               300       0.2030       0.1851       0.1247       0.2422   -0.0571
             1,000       0.1460       0.1409       0.0959       0.1447   -0.0038
             3,000       0.1352       0.1326       0.0881       0.1085   +0.0240
            10,000       0.1301       0.1292       0.0864       0.0956   +0.0336
            60,000       0.1270       0.1278       0.0847       0.0913   +0.0365
(71 s; the 60,000 row is one fit, repeated)
breast cancer: 30 measurements (log, then standardised by the training rows); 369 training and 200 test
    tumours; 20 random subsets per size
    training rows  naive Bayes          LDA          QDA     logistic   NB - logistic
               10       0.0970       0.1130       0.0900       0.0993   -0.0023
               20       0.0553       0.0848       0.0595       0.0760   -0.0207
               40       0.0413       0.0458       0.0377       0.0470   -0.0057
               80       0.0363       0.0358       0.0355       0.0320   +0.0042
              160       0.0308       0.0378       0.0323       0.0313   -0.0005
              369       0.0300       0.0400       0.0350       0.0200   +0.0100
one test error on 200 tumours is 0.005; settings fixed at every size (generative reg 0.1, QDA 1.0; logistic
    lambda 0.001), not re-tuned
Actual output from this lesson's run.
```

What it means

On MNIST the like-for-like pair (LDA and logistic regression, both linear) shows the pattern clearly: LDA is ahead by 0.1239 at 30 images and 0.0571 at 300, the two are level at 1,000 (difference -0.0038), and logistic regression is ahead from 3,000 on, by 0.0365 at 60,000. QDA is best at every size (0.0847 at 60,000), but it is not a like-for-like comparison: one covariance per digit draws curved boundaries, a richer model than either linear one.

What it means

The breast-cancer table is too small to say much: naive Bayes is clearly ahead at 20 rows (0.0553 against 0.0760), and logistic regression ends lowest at 369 (0.0200 against 0.0300), but on 200 test tumours that is 4 mistakes against 6, from a single fit.



Top: learning curves on MNIST and breast cancer (dashed: generative; solid: discriminative); digits drawn from QDA's p(x | digit). Bottom: accuracy with half of each image hidden; spotting unusual images; logistic regression's confidence on them.

WHY CHOOSE THIS?

Generative models learn faster from few examples.

WHY NOT THIS? (TRADEOFFS)

Their assumptions (Gaussian classes) limit how good they can get.

4. Generating: drawing digits from p(x | digit)

DEFINITION — start here, no prior knowledge assumed

A generative model can produce new data: draw from each digit's Gaussian (QDA, in 50 PCA numbers) and map back to pixels. Section 2's logistic regression judges whether each sample looks like the digit it was drawn as.

REAL-WORLD SCENARIO

A forger who has studied many signatures can produce a new one; a clerk who only checks signatures cannot.

MATHEMATICS

$$x = \mu_c + L\,\varepsilon, \quad L\,L^T = \Sigma_c, \quad \varepsilon \sim N(0, I) \;\Rightarrow\; x \sim N(\mu_c, \Sigma_c)$$

IMPLEMENTATION

```
def sample_class(model, c, n, rng):
    L = np.linalg.cholesky(model["cov"][c])
    return model["mu"][c] + rng.standard_normal((n, len(L))) @ L.T
```

ACTUAL OUTPUT

```
QDA with reg 1.0: 1,000 sampled images; section 2's logistic regression names the intended digit for 0.5410
   per digit: 0 0.68, 1 0.01, 2 0.51, 3 0.79, 4 0.39, 5 0.51, 6 0.64, 7 0.73, 8 0.57, 9 0.58
QDA with reg 0.001: 1,000 sampled images; section 2's logistic regression names the intended digit for 0.8620
   per digit: 0 0.97, 1 0.85, 2 0.81, 3 0.92, 4 0.77, 5 0.77, 6 0.94, 7 0.90, 8 0.83, 9 0.86
each digit's own total variance in the 50 numbers: 0 40.5, 1 18.2, 2 38.4, 3 35.1, 4 31.7, 5 37.3, 6 33.2, 7
    28.9, 8 34.8, 9 30.4; reg 1.0 adds 50 x 1.0 = 50.0
for comparison, 1,000 real test digits squeezed through the same 50 PCA numbers: 0.8600
a logistic regression has no p(x | digit) to draw from: it can only label images it is given
Actual output from this lesson's run.
```

What it means

The first attempt reused section 3's setting, reg 1.0, and failed for one digit: samples of 1 were recognised 0.01 of the time. The script prints why: a 1's own total variance in the 50 numbers is 18.2, and reg 1.0 adds 50 more in every direction, so its samples drown in noise. With reg 0.001 the samples are recognised 0.8620 of the time, as often as real test digits squeezed through the same 50 numbers (0.8600). In the figure they look like digits, blurred and blotchy: a Gaussian cannot represent a stroke, only an average shape with correlated noise around it.

WHY CHOOSE THIS?

New data for free, from the same model that classifies.

WHY NOT THIS? (TRADEOFFS)

Gaussian samples are blurry; better generators (GANs, from tomorrow) are much harder to train.

5. Missing pixels

DEFINITION — start here, no prior knowledge assumed

Hide half of every test image. A Gaussian model simply leaves the hidden pixels out: the **marginal** of a Gaussian over the seen pixels is the Gaussian with those rows and columns of the mean and covariance. Logistic regression needs every pixel, so the hidden ones must be filled in (with 0 or the average image), or a new model trained for that exact pattern.

REAL-WORLD SCENARIO

A doctor who knows how symptoms go together can still diagnose when one test result is missing.

IMPLEMENTATION

```
def log_joint(model, X, keep=None):
    idx = np.arange(X.shape[1]) if keep is None else keep
    return np.stack([log_gauss(X[:, idx], m[idx], S[np.ix_(idx, idx)]) + np.log(p)
                     for m, S, p in zip(model["mu"], model["cov"], model["prior"])], 1)
```

ACTUAL OUTPUT

```
   pattern             LDA, hidden pixels left out  logistic, hidden = 0  logistic, hidden = average  logistic
       retrained on that half
   top half seen                            0.7771                0.3923                      0.4669
       0.7749
   bottom half seen                         0.7625                0.4303                      0.5767
       0.7390
   left half seen                           0.7936                0.5032                      0.5720
       0.8005
   right half seen                          0.7870                0.4795                      0.5615
       0.7528
the retrained column needs a new model for every pattern of missing pixels; LDA uses the one model it already
    has
Actual output from this lesson's run.
```

What it means

With the top half seen, LDA scores 0.7771; logistic regression with the hidden half filled scores 0.3923 (zeros) or 0.4669 (average image). A logistic regression retrained on top halves only matches LDA (0.7749), but it is a new model, and it works only for that one pattern. Across the four patterns, LDA is within about 0.04 of the retrained models (ahead on three, behind on the left half), using the one model it already had.

WHY CHOOSE THIS?

Handles any pattern of missing inputs, exactly, with no retraining.

WHY NOT THIS? (TRADEOFFS)

Exact only when the Gaussian model is right; it does nothing to recover the hidden pixels' detail.

6. Knowing what it does not know

DEFINITION — start here, no prior knowledge assumed

A generative model gives log p(x): how ordinary an image is under the model. A discriminative model gives only p(class | x), whose top probability is often used as its 'confidence'. Both are tested as detectors of three kinds of image that are not a single upright digit: upside-down digits, uniform random noise, and two digits overlaid.

REAL-WORLD SCENARIO

A guard who knows what staff look like notices a stranger; one trained only to tell two departments apart sorts the stranger into one of them.

IMPLEMENTATION

```
def log_px(model, X):
    J = log_joint(model, X); m = J.max(1)
    return m + np.log(np.exp(J - m[:, None]).sum(1))
```

ACTUAL OUTPUT

```
score: generative, -log p(x) under section 2's LDA; discriminative, 1 - the logistic regression's top
    probability
   unusual images        AUC -log p(x)  AUC 1 - confidence  logistic's average confidence on them
   digits upside down           0.7259              0.7061  0.9148
   random noise                 1.0000              0.0110  1.0000
   two digits overlaid          0.8334              0.5935  0.9504
for comparison, its average confidence on normal test digits: 0.9702
Actual output from this lesson's run.
```

What it means

Random noise is the starkest case: log p(x) separates it perfectly (AUC 1.0000), while logistic regression is more confident on noise (1.0000 on average) than on real digits (0.9702), so its confidence ranks noise as the most normal of all (AUC 0.0110). Overlaid digits: 0.8334 against 0.5935. Upside-down digits are hard for both (0.7259 and 0.7061): many still look like plausible digits.

WHY CHOOSE THIS?

log p(x) flags inputs unlike the training data.

WHY NOT THIS? (TRADEOFFS)

Only as well as the model of x: a Gaussian is a crude model of digits.

Interview preparation — technical questions

Q: What is the difference between a generative and a discriminative classifier?

A: The generative one models p(x | class) and p(class) and uses Bayes' rule; the discriminative one models p(class | x) directly.

Q: When would you pick the generative one?

A: With few labels (LDA 0.2922 against 0.3702 error at 100 images), missing inputs, or when you need p(x).

Q: Is LDA a linear classifier?

A: Yes: its posterior is a softmax of linear scores; checked here to 8.0e-14.

Q: How do you classify with missing features in a Gaussian model?

A: Drop the missing rows and columns of the mean and covariance: the marginal is exact.

Q: Can a classifier's confidence detect unusual inputs?

A: Not reliably: logistic regression was 1.0000 confident on random noise.

Interview preparation — theoretical questions

Q: Why does the discriminative model win with lots of data?

A: It makes no assumption about how x is distributed, so wrong assumptions cannot limit it.

Q: Why does the generative model win with little data?

A: Its assumptions supply structure that a few examples cannot; the price is a worse limit when they are wrong.

Q: What does regularising a covariance do?

A: Adds the same variance in every direction: needed to invert it, but too much drowns a class's own shape (section 4).

Q: What did Ng and Jordan show?

A: That naive Bayes approaches its best error with far fewer examples than logistic regression, which ends lower.

Key takeaway and what's next

KEY TAKEAWAY

A generative classifier learns how the data is made and gets the boundary for free; a discriminative one learns only the boundary. Measured here: the generative model wins with little data, the discriminative one with a lot, and only the generative model can sample, cope with missing inputs and notice that an input is unlike anything it has seen. Its weakness is its assumptions: Gaussian digits are blurry, which is where the rest of the week's generators come in.

- Same family: LDA's posterior is a linear softmax (8.0e-14); accuracy 0.8798 against logistic regression's 0.8917.
- Learning curves: LDA ahead up to 300 images, level at 1,000, logistic regression ahead from 3,000.
- Half of each image hidden: LDA 0.7771, logistic regression with zeros 0.3923.
- Random noise: log p(x) AUC 1.0000; logistic regression 1.0000 confident on it.

Suggested watch order, if starting from zero

- 1. (ML 1.5) Generative vs discriminative models (mathematicalmonk) — the two families defined, with the probabilities behind them.
- 2. IAML2.23: Generative vs. discriminative learning (Victor Lavrenko) — the trade-offs between them, in a short lecture.

Day 134 preview

- **A GAN from scratch**: a generator and a discriminator trained against each other, with no formula for p(x).